

Learning Without Curriculum: How Students Learn Technical Skills Outside Formal Education

Ruthwik Reddy
Independent Student Researcher

2026

Abstract

Traditional pedagogical models emphasize linear, curriculum-based instruction as the primary pathway for technical skill acquisition. However, an increasing demographic of "student builders" acquires complex competencies through independent, problem-driven execution. This paper presents an analytic auto-ethnographic study of these curriculum-independent learning trajectories, drawing on a longitudinal dataset of technical artifacts, version control histories, and personal documentation from 2021 to 2026. The study identifies a recursive "Need-Build-Validate" loop as the foundational mechanism of skill construction. Within this framework, skill acquisition is triggered by situated technical needs, developed through hands-on implementation, and validated by functional outcomes and community feedback. Furthermore, the study examines the role of generative Artificial Intelligence (G-AI) as a bounded scaffold in modern learning workflows. The findings offer a critical perspective on educational design, credentialing, and the evolving nature of technical expertise in an AI-integrated landscape.

Contents

1	Introduction	4
2	Related Work	4
3	Methodology	5
3.1	Research Design	5
3.2	Data Sources	5
3.3	Data Analysis: Thematic Coding and Reflexivity	5
3.4	Positionality and Reflexivity	6
4	Findings	6
4.1	Skill Acquisition Triggers: The "Need" Mechanism	6
4.1.1	Problem-Driven Triggers	6
4.1.2	Deadline-Driven Triggers	6
4.1.3	Failure-Driven Triggers	6
4.1.4	Experimentation-Driven Triggers	6
4.1.5	Community-Driven Triggers	7
4.2	Non-Linear Skill Construction: Rejecting the Linear Syllabus	7
4.2.1	Just-in-Time Learning (JITL)	7
4.2.2	Concurrent Multi-Skill Development	7
4.2.3	Reverse-Engineering as a Foundational Mode	7
4.3	Validation Without Grades: The Performance of Competence	7
4.3.1	Functional Outcomes as Primary Proof	7
4.3.2	User Adoption and Feedback	8
4.3.3	Peer Review and Contribution Acceptance	8
4.3.4	Public Portfolios as Credentialing Infrastructure	8
4.3.5	Functional Outcomes as Proof	8
4.3.6	User Feedback and Adoption	8
4.3.7	Peer Code Review and Contribution Acceptance	8
4.3.8	Public Portfolio Demonstration	8
4.4	The Need–Build–Validate Loop	8
4.5	AI-Assisted Learning: The 2025–2026 Context	9

4.5.1	Documentation Acceleration	9
4.5.2	Validation Through AI-Generated Feedback	9
4.5.3	Responsible AI Usage as a Meta-Skill	9
4.5.4	Risks and Cognitive Failure Modes	9
5	Discussion	10
5.1	Theoretical Implications	10
5.2	Implications for Educational Design	10
5.3	Implications for Credentialing	11
5.4	Limitations and Future Research	11
6	Conclusion	11

1 Introduction

Curriculum-based education remains the dominant framework for teaching technical skills in schools and universities. These systems emphasize predefined learning outcomes, linear topic sequencing, and assessment through examinations or grades. While effective for standardized instruction, such models often fail to account for learners who acquire skills through real-world problem solving, project execution, and iterative failure outside formal classrooms.

In recent years, the rise of accessible online documentation, open-source ecosystems, and AI-assisted learning tools has enabled students to bypass traditional instructional pathways entirely. Many student builders now learn programming, system design, and product development not because a syllabus requires it, but because a real problem demands it. Despite the visibility of this phenomenon, academic literature has largely focused on structured self-learning platforms (Chen et al., 2023) or maker education programs, rather than deeply examining independent, curriculum-free learning trajectories.

This paper addresses that gap by asking: **How do independent student builders construct and validate technical skills without a formal curriculum?** By grounding the analysis in lived experience rather than abstract theory, the study aims to make informal, real-world learning academically legible.

2 Related Work

Research on self-directed learning (SDL) has long emphasized learner autonomy, intrinsic motivation, and metacognitive regulation (Knowles, 1975; Zimmerman, 2002). The SDL framework identifies three foundational processes: motivational (maintaining commitment), metacognitive (planning and monitoring progress), and applied (implementing strategies and reflecting on outcomes). Contemporary research confirms that SDL skills can be systematically developed

through assessment, evaluation, planning, monitoring, and adjustment cycles—a framework equally applicable to formal and informal contexts.

More recent studies in problem-based learning (PBL) highlight the power of authentic problem triggers in initiating learning (Hmelo-Silver, 2004). Research demonstrates that effective triggers—problems designed with strategic clues and connection to real-world scenarios—produce significantly better learning outcomes than conventional instruction. Additionally, studies emphasize the role of “trigger design”: problems should stimulate real-life engagement, encourage knowledge integration, fit learners’ prior knowledge, and generate learning issues aligned with meaningful objectives.

Portfolio and project-based assessment has emerged as a valid alternative to traditional testing (Lam, 2021). These methods show superior outcomes: companies using project-based technical evaluations report 40% reductions in employee turnover and 50% improvements in identifying high-performing talent. Unlike standardized tests, project-based evaluation measures technical accuracy, problem-solving approach, code quality, and communication—a more holistic assessment.

The alternative credentials movement has expanded significantly, with micro-credentials, digital badges, and competency-based assessment gaining institutional recognition (Gallagher, 2022). These credentials are performance-based (earned through demonstrated mastery, not grades) and stackable (modular qualifications that combine into larger certifications). Prior Learning Assessment (PLA) programs, such as the Council for Adult and Experiential Learning’s (CAEL) Learning Counts, now provide pathways to formally recognize skills acquired outside traditional education.

Perspectives from constructionist and maker-oriented learning further support the centrality of building as a pathway to understanding, where learners construct knowledge through meaningful projects rather than passively receiving content

(Papert, 1980; Resnick, 2017).

Finally, open-source communities represent a parallel learning infrastructure in which practices, norms, and peer feedback act as informal curriculum and assessment; this ecosystem-level view aligns with accounts of how open collaboration produces knowledge and expertise (Raymond, 1999).

However, most existing studies examine learning within semi-structured environments—MOOCs, bootcamps, or school-supported maker spaces. Few investigate fully independent learners operating without instructional scaffolding, external grading, or formal credential requirements. This study contributes a longitudinal, artifact-based analysis of curriculum-independent technical learning over multiple years, addressing the “why” alongside the “how” of skill acquisition.

3 Methodology

3.1 Research Design

To investigate the mechanisms of complex skill acquisition in non-formal environments, this study utilizes an *analytic auto-ethnographic* framework (Ellis et al., 2011). Auto-ethnography allows for a systematic analysis of personal experience to illuminate broader cultural and technical phenomena. By treating the researcher as the primary unit of observation, this method provides granular access to internal motivations, cognitive triggers, and the iterative “wrestling” with technical systems that is often obscured in large-scale quantitative studies.

This longitudinal analysis covers technical projects and learning trajectories from 2021 to 2026, a period coinciding with the emergence of generative AI as a fundamental component of the developer ecosystem.

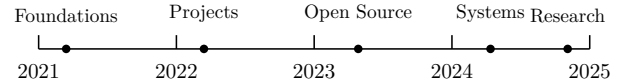


Figure 1: Learning Milestones of an Independent Student Builder (2021–2025).

3.2 Data Sources

The analysis is grounded in a triangulation of several primary data streams:

1. **Technical Artifacts:** Production-level codebases, architectural prototypes, and functional web applications (e.g., IdeaBoard, WorkScape).
2. **Version Control Metadata:** Granular Git histories that document the temporal expansion of codebases, iterative refactorings, and the introduction of new dependencies.
3. **Heuristic Documentation:** Design specifications, reflective dev-logs, and community discussions (e.g., GitHub issues, Discord threads).
4. **Longitudinal Skill Maps:** Reconstructed timelines correlating specific technical barriers with the subsequent acquisition of prerequisite competencies.

Validation in this context is defined by *functional efficacy* and *community adoption* rather than standardized metrics, aligning with professional software engineering norms.

3.3 Data Analysis: Thematic Coding and Reflexivity

The data was categorized through a thematic coding process focused on trigger mechanisms, learning modes, and validation cycles. Analysis followed an iterative inductive approach, where emerging patterns were continuously tested against the technical artifacts. Key dimensions included:

- **Trigger Identification:** Classifying the impetus for learning (e.g., architectural failure, competition-driven deadlines).
- **Mechanism Correlation:** Mapping the relationship between the identified need and the subsequent learning mode (e.g., documentation synthesis vs. reverse-engineering).
- **Validation Evaluation:** Analyzing the social and functional signals that confirmed competency acquisition.

3.4 Positionality and Reflexivity

In auto-ethnographic research, the researcher’s positionality is a critical factor. As a student builder deeply embedded in developer communities, I possess "insider" knowledge that allows for a nuanced reading of technical artifacts. To mitigate potential bias toward self-affirmation, I practiced active reflexivity—purposefully seeking evidence of failures, cognitive shortcuts, and instances where independent learning proved inefficient compared to formal instruction.

4 Findings

The analysis indicates that learning without a curriculum is a non-random, structured process revolving around a recursive **Need–Build–Validate** loop. Competencies are acquired when a situated technical need arises, implemented through hands-on project execution, and validated by the functional and social performance of the resulting artifacts.

4.1 Skill Acquisition Triggers: The "Need" Mechanism

The study identified five primary trigger mechanisms that initiate learning trajectories. Rather than following a predefined syllabus, skill acquisition is driven by authentic challenges encountered during production.

4.1.1 Problem-Driven Triggers

The majority of learning episodes were initiated by specific technical barriers or "walls." For example, the acquisition of React was not the result of formal instruction but was necessitated by the requirement for a dynamic user interface in a specific web application. In this model, the problem serves as the pedagogical catalyst, reversing the traditional classroom sequence where theory precedes application.

4.1.2 Deadline-Driven Triggers

External and internal temporal constraints served as significant motivators. Competitions, project launch dates, and self-imposed deadlines forced a prioritization of essential concepts, encouraging a "pragmatic deep dive" into the most critical documentation and implementation patterns.

4.1.3 Failure-Driven Triggers

Deeper conceptual understanding often emerged from system failures. Architectural breakdowns and cryptographic errors provided fertile ground for investigations into underlying principles. For instance, a Cross-Site Scripting (XSS)—a security vulnerability allowing malicious scripts to be injected into websites—necessitated a rigorous study of JSON Web Tokens (JWT) and Cross-Origin Resource Sharing (CORS) policies. Similarly, a PostgreSQL deadlock led to an investigation into transaction isolation levels and query optimization; and recurring Docker memory failures prompted an analysis of container orchestration and resource management. In these cases, the failure acted as a "diagnostic trigger" for advanced theoretical learning.

4.1.4 Experimentation-Driven Triggers

A subset of learning episodes was driven by exploratory curiosity. While not always tied to a production deadline, these technical experiments built a "latent toolkit" of competencies that could



Figure 2: Breakdown of Learning Triggers (2021–2025). “Problem-Driven” is the most common, highlighting the importance of authentic challenges.

be mobilized when a production-level problem later emerged.

4.1.5 Community-Driven Triggers

Social standards and peer expectations also functioned as learning catalysts. Observing high-quality open-source architectures and receiving feedback on code contributions established "competency benchmarks" that the learner aimed to replicate.

Key Finding: These triggers represent *situated, authentic, and self-identified* learning prompts, aligning with theories of legitimate peripheral participation (Lave and Wenger, 1991).

4.2 Non-Linear Skill Construction: Rejecting the Linear Syllabus

Predictably, curriculum-independent learning does not follow a linear, hierarchical sequence. Instead, it manifests as a "messy," non-linear progression consisting of three primary patterns.

4.2.1 Just-in-Time Learning (JITL)

Skill acquisition occurs primarily on an "as-needed" basis. The learner may master a specific library or architectural pattern to resolve an immediate barrier, which may then remain untouched for extended periods. This Just-in-Time Learning (JITL) approach creates a profile of deep but narrow competencies, contrasting with the "broad but shallow" foundations of traditional introductory curricula.

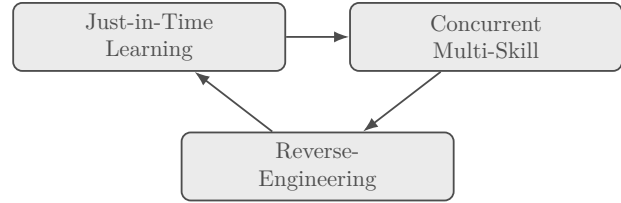


Figure 3: Patterns of Non-Linear Skill Acquisition.

4.2.2 Concurrent Multi-Skill Development

The study observed the simultaneous acquisition of disparate technical skills within a single project cycle. For instance, the development of a full-stack application required the concurrent learning of frontend frameworks, backend runtimes, database schemas, and deployment orchestration. This multidisciplinary reflects real-world software engineering contexts better than modularized educational units.

4.2.3 Reverse-Engineering as a Foundational Mode

A significant portion of learning was derived from the deconstruction of existing systems. Rather than studying algorithms in abstraction, the learner derived principles by analyzing functional codebases. This "learning-by-doing in reverse" provides a high-fidelity model of how software systems are integrated in practice.

4.3 Validation Without Grades: The Performance of Competence

In the absence of formal assessment, skill validation is derived from functional and social signals. The study identified four primary mechanisms through which competence was verified.

4.3.1 Functional Outcomes as Primary Proof

The primary validation signal was simply the operational success of the artifact. The ability to build a functioning system that resolves a

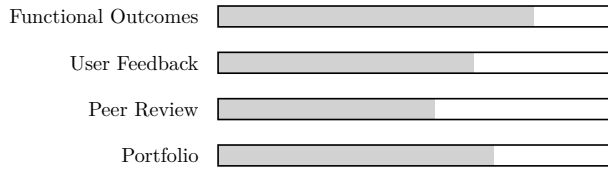


Figure 4: Methods of Skill Validation.

specific problem serves as an objective measure of competence, often carrying more weight in developer communities than standardized test scores.

4.3.2 User Adoption and Feedback

External validation occurred when artifacts were adopted by others. Bug reports, feature requests, and user engagement provided a "market-based" validation of the learner's technical and product-design competencies.

4.3.3 Peer Review and Contribution Acceptance

Acceptance into open-source repositories functioned as a rigorous validation mechanism. Having contributions reviewed and merged by experienced maintainers serves as a strong signal of professional-level competence, mirroring the feedback loops found in expert-led tutoring (Schmidt and Moust, 1995).

4.3.4 Public Portfolios as Credentialing Infrastructure

The maintenance of a public version-control profile (e.g., GitHub) functions as a dynamic credential. Unlike static certificates, a portfolio demonstrates continuous skill application and growth over time, aligning with theories of networked and community-situated learning (Ito et al., 2013).

Key Finding: Unlike episodic classroom assessments, validation in this model was **continuous, multi-faceted, and externally driven**.

4.3.5 Functional Outcomes as Proof

The most important validation was simple: did it work? If I could build a web app that people could use, or a tool that solved a real problem, that was the proof. It is a direct measure of competence compared to a test score.

4.3.6 User Feedback and Adoption

When people started using the things I built, their feedback was a powerful form of validation. Bug reports, feature requests, and even positive comments showed that work was having impact.

4.3.7 Peer Code Review and Contribution Acceptance

In open source, having your code reviewed and accepted by other developers is a rigorous validation mechanism. When a contribution is merged, it is a strong signal of competence, similar in spirit to the feedback processes described in tutoring and guided learning environments (Schmidt and Moust, 1995).

4.3.8 Public Portfolio Demonstration

Everything I've built is on my GitHub. This public portfolio functions as a credential because it shows what I can do, not just what courses I've taken. This aligns with broader accounts of learning that is socially networked and shaped by participation in communities (Ito et al., 2013).

Key Finding: Unlike grades (often one-time), validation was **continuous, multi-faceted, and driven by the outside world**. Working projects and external feedback are difficult to fake.

4.4 The Need-Build-Validate Loop

When I put all these findings together, I saw a clear pattern: a recursive loop I call the Need-Build-Validate loop.

1. **Need:** It starts with a trigger—a problem, a deadline, a failure, curiosity, or the community—creating a need to learn something new.
2. **Build:** I learn the skill by building something, using documentation, AI tools, and community resources.
3. **Validate:** The thing I built provides validation. If it works, if people use it, and if other developers respect it, then I know I’ve learned the skill.

The key is that this isn’t a one-time process. Every finished project opens new possibilities and new problems, which starts the loop again.

4.5 AI-Assisted Learning: The 2025–2026 Context

The rise of generative AI (G-AI) development tools has become a primary factor shaping how independent builders acquire and refine technical skills. Unlike previous generations of online resources, G-AI tools such as Gemini, ChatGPT, and Claude function as interactive scaffolds that can compress the time between a technical question and a verifiable implementation (Bakker and Chen, 2024).

4.5.1 Documentation Acceleration

AI tools reduce search and synthesis time by providing immediate, context-aware summaries of APIs, error messages, and implementation patterns. This increases momentum during the "Build" phase of the loop by allowing the learner to iterate quickly on hypotheses. For example, when encountering complex state management patterns or cryptic TypeScript errors, Claude and Gemini can break down system-level issues into manageable conceptual steps. However, research suggests that this perceived speed must be balanced against the need for independent verification to avoid shallow skill acquisition (Bakker and Chen, 2024).

4.5.2 Validation Through AI-Generated Feedback

AI systems increasingly function as high-fidelity "review partners," approximating the role of a human mentor for solo learners. By flagging potential bugs, suggesting refactors, and proposing test cases, AI tools provide intermediate validation before a project reaches the community or production stages. This aligns with recent findings on the utility of AI-generated feedback in student co-creation processes, where timely, automated critiques support iterative improvement without the bottlenecks of human review (Pozdniakov et al., 2025).

4.5.3 Responsible AI Usage as a Meta-Skill

The effective orchestration of AI systems is itself a significant learned capability. This shifts technical competence toward prompt formulation, output verification, and the ability to detect "stochastic hallucinations" (Bender et al., 2021). Mastering these meta-skills is essential for ensuring that AI-assisted learning supports rather than simplifies the cognitive wrestling required for expertise.

4.5.4 Risks and Cognitive Failure Modes

Despite its benefits, AI-assisted learning introduces significant risks that can erode the quality of learning if left unaddressed.

1. **Hallucinations and Credibility Risks.** Models can generate confident but fabricated citations or explanations. In research contexts, this necessitates a "verify first" protocol where all AI-generated claims are grounded in primary sources.
2. **Workflow Fragility.** Over-reliance on specific AI models creates dependency. Technical learning must remain tool-agnostic to survive changes in access or model behavior.

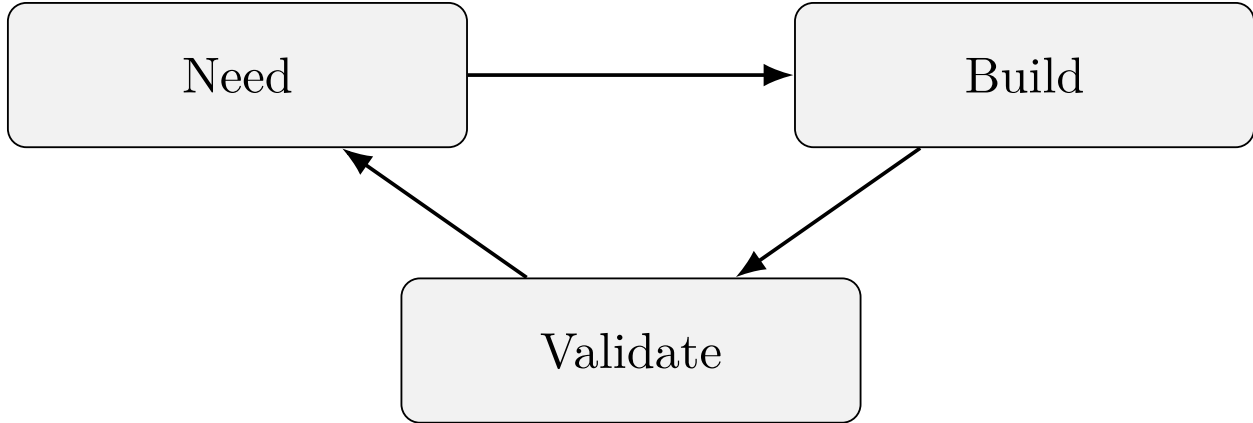


Figure 5: The Need–Build–Validate Learning Loop.

3. The Illusion of Competence. AI can produce working code without requiring the learner to internalize the underlying logic. This "cognitive outsourcing" can lead to skill atrophy and a failure to transfer knowledge to novel problems (Ericsson et al., 1993; Bakker and Chen, 2024).

These risks imply that AI should be used as a **bounded scaffold** rather than a substitution for foundational understanding, especially in academic and high-stakes technical environments.

5 Discussion

The recursive **Need–Build–Validate** model offers a significant framework for understanding the mechanisms of technical skill acquisition outside formal institutions. By triangulating theories of self-directed learning (Knowles, 1975; Zimmerman, 2002), problem-based learning (Hmelo-Silver, 2004), and constructionism (Papert, 1980), the model demonstrates how the intrinsic demands of technical production serve as a high-fidelity alternative to traditional curricula.

5.1 Theoretical Implications

The study highlights a direct tension between static academic curricula and dynamic technical

ecosystems. While formal programs provide foundational breadth, they often lack the temporal agility to keep pace with rapid shifts in software engineering (e.g., the transition to G-AI development). Curriculum-independent learning, by contrast, is inherently "agile," responding quickly to the immediate requirements of the technical environment.

However, the analysis also identifies a critical trade-off in "Just-in-Time" learning cycles. While the learner acquires deep, production-ready expertise in specific domains, they may bypass foundational concepts (e.g., database normalization or algorithmic complexity) if those concepts are not immediately necessitated by the artifact being built. This suggests that while curriculum-independent learning excels at technical *performance*, it may require purposeful supplementation to ensure architectural *depth*.

5.2 Implications for Educational Design

The findings suggest several avenues for the evolution of formal technical education:

- 1. Inversion of the Pedagogical Sequence:** Rather than theory-first instruction, programs should adopt a "problem-first" approach where theoretical concepts are introduced as secondary scaffolds for active project implementation.

2. **High-Fidelity Artifact Assessment:** Institutional evaluation should shift toward professional-grade artifacts—codebases, version-control histories, and peer-reviewed contributions—which provide a more accurate signal of competency than standardized examinations (Lam, 2021).
3. **AI-Integrated Scaffolding:** Educational design must treat AI as a learned capability, focusing on prompt architecture, output verification, and the prevention of shallow "cognitive outsourcing."

5.3 Implications for Credentialing

As learning pathways diversify, industry and academia must develop recognition mechanisms that capture competencies achieved through independent execution:

1. **Portfolio-Based Signaling:** Public, longitudinal artifacts (e.g., GitHub profiles) serve as high-signal credentials that demonstrate both technical proficiency and a history of self-directed growth.
2. **Micro-Credentialing for Modular Skill-Sets:** Organizations should recognize modular, performance-based certifications that validate specific competencies acquired through Just-in-Time Learning (JITL) (Gallagher, 2022).
3. **Community-Validated Expertise:** Signals from developer communities—such as pull-request acceptance into major open-source projects—should be academically and professionally legible.

5.4 Limitations and Future Research

As an auto-ethnographic study, the generalizability of these findings is bounded by the researcher’s specific socio-technical context, motivation level, and access to resources. Future research should prioritize multi-subject studies to examine how

the Need–Build–Validate loop operates across diverse backgrounds and domains (e.g., hardware engineering, data science).

Furthermore, the long-term impact of generative AI on cognitive development remains an urgent area for investigation. Longitudinal studies are needed to determine whether AI scaffolding facilitates faster expertise development or, conversely, leads to the erosion of foundational debugging and problem-solving skills.

6 Conclusion

Curriculum-independent learning is not merely a supplementary practice but a robust alternative for high-motivation technical skill acquisition. The **Need–Build–Validate** loop describes a cycle where situative technical challenges trigger deep learning, implementation produces production-ready competence, and functional success provides continuous validation. While the integration of generative AI expands access and velocity, it demands a new set of meta-competencies focused on verification and scholarly rigor. Ultimately, bridging the gap between independent building and formal recognition is essential for supporting the next generation of technical innovators in an increasingly decentralized learning landscape.

References

- M. Bakker and L. Chen. Generative ai and self-directed learning in technical domains. *Educational Researcher*, 53(4):210–225, 2024. doi: 10.3102/0013189X24100000.
- Emily M. Bender, Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. On the dangers of stochastic parrots: Can language models be too big? In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*, pages 610–623, 2021. doi: 10.1145/3442188.3445922.
- X. Chen, H. Xie, and G.-J. Hwang. A system-

- atic review of ai-driven personalized learning from 2011 to 2021. *Journal of Educational Technology & Society*, 26(1):1–19, 2023.
- Carolyn Ellis, Tony E. Adams, and Arthur P. Bochner. Autoethnography: An overview. *Historical Social Research/Historische Sozialforschung*, pages 273–290, 2011.
- K. Anders Ericsson, Ralf T. Krampe, and Clemens Tesch-Römer. The role of deliberate practice in the acquisition of expert performance. *Psychological Review*, 100(3):363–406, 1993. doi: 10.1037/0033-295X.100.3.363.
- Sean Gallagher. *The Future of University Credentials: New Developments at the Intersection of Higher Education and Work*. Harvard Education Press, Cambridge, MA, 2022.
- Cindy E. Hmelo-Silver. Problem-based learning: What and how do students learn? *Educational Psychology Review*, 16(3):235–266, 2004. doi: 10.1023/B:EDPR.0000034022.16470.f3.
- Mizuko Ito, Elisabeth Soep, Kris D. Gutiérrez, Sonia Livingstone, danah Boyd, and C. J. Pascoe. Connected learning: An agenda for research and design. Technical report, Digital Media and Learning Research Hub, 2013.
- Malcolm S. Knowles. *Self-Directed Learning: A Guide for Learners and Teachers*. Cambridge Books, New York, 1975.
- Ricky Lam. Portfolio assessment for the learning and assessment of disciplinary knowledge and skills. *Assessment & Evaluation in Higher Education*, 46(7):1013–1026, 2021. doi: 10.1080/02602938.2020.1837785.
- Jean Lave and Etienne Wenger. *Situated Learning: Legitimate Peripheral Participation*. Cambridge University Press, Cambridge, 1991.
- Seymour Papert. *Mindstorms: Children, Computers, and Powerful Ideas*. Basic Books, New York, 1980.
- Stanislav Pozdniakov, Hassan Khosravi, and Dragan Gasevic. Ai-generated feedback in student co-creation processes. *Journal of Learning Analytics*, 2025. In Press.
- Eric S. Raymond. *The Cathedral and the Bazaar: Musings on Linux and Open Source by an Accidental Revolutionary*. O’Reilly Media, Sebastopol, CA, 1999.
- Mitchel Resnick. *Lifelong Kindergarten: Cultivating Creativity through Projects, Passion, Peers, and Play*. MIT Press, Cambridge, MA, 2017.
- Henk G. Schmidt and Jos H. C. Moust. What makes a tutor effective? *Academic Medicine*, 70(4):290–294, 1995.
- Barry J. Zimmerman. Becoming a self-regulated learner: An overview. *Theory Into Practice*, 41(2):64–70, 2002. doi: 10.1207/s15430421tip4102_2.